

Module 09 — SQL Reporting & IA

Bootcamp Data Analyst — From Zero to Hero | Niveau Débutant · Acte III

Ce que tu seras capable de faire à la fin de ce module

- Écrire des **sous-requêtes** pour filtrer ou comparer des données dynamiquement
- Organiser des requêtes complexes avec les CTEs (`WITH`)
- Créer de la logique conditionnelle avec `CASE WHEN`
- Calculer des **classements** et des **comparaisons temporelles** avec les window functions (`RANK` , `ROW_NUMBER` , `LAG`)
- Utiliser l'IA (Claude / ChatGPT) pour écrire du SQL plus vite — et vérifier ses résultats

 **Durée estimée** : 50 minutes

 **Prérequis** : Modules 06, 07, 08 — `SELECT` , `GROUP BY` , `JOIN` maîtrisés

 **Environnement** : sqliteonline.com — mêmes 3 tables que M08

1. Pourquoi ce module ?

Tu sais maintenant filtrer (`WHERE`), résumer (`GROUP BY`) et relier des tables (`JOIN`). Mais un DA junior doit aller plus loin : **produire des rapports**.

Un rapport, c'est souvent :

- "Affiche-moi les clients au-dessus de la moyenne" → sous-requête

- "Étape par étape : calcule le CA, puis classe les agents" → CTE
- "Ajoute une colonne Segment selon le montant" → CASE WHEN
- "Compare le CA de ce mois avec le mois dernier" → window function

Et en 2024, un DA efficace sait aussi **utiliser l'IA** pour écrire du SQL — sans se laisser tromper par les hallucinations.



Scénario fil rouge : le DG de l'ANSUT-CI te demande le rapport mensuel complet des ventes. Tu vas construire ce rapport pièce par pièce, en utilisant tous les outils de ce module.

2. Sous-requêtes — une requête dans une requête

Une **sous-requête** (ou subquery) est une requête **SELECT** imbriquée dans une autre. Elle s'exécute en premier et son résultat est utilisé par la requête principale.

```
SELECT ...  
FROM ...  
WHERE colonne [opérateur] (SELECT ... FROM ...);  
                        ↑  
                sous-requête : s'exécute d'abord
```

2.1 Sous-requête scalaire — une seule valeur

Question : Quels clients ont un CA total supérieur à la moyenne des clients ?

SQL

```
SELECT
  c.nom,
  SUM(t.montant) AS ca
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.id_client, c.nom
HAVING SUM(t.montant) > (
  SELECT AVG(ca)
  FROM (SELECT SUM(montant) AS ca FROM transactions GROUP BY id_client)
)
ORDER BY ca DESC;
```

NOM	CA
Kouassi Ama	5 000

La sous-requête calcule d'abord la moyenne des CA par client (2 583 FCFA). La requête principale filtre ensuite les clients au-dessus. Sans sous-requête, il faudrait faire le calcul manuellement.

2.2 Sous-requête avec IN — une liste de valeurs

Question : Quels clients ont acheté le forfait internet ?

SQL

```
SELECT c.nom, c.ville
FROM clients c
WHERE c.id_client IN (
  SELECT DISTINCT id_client
  FROM transactions
  WHERE id_forfait = 3
);
```

NOM	VILLE
Kouassi Ama	Abidjan
Ouédraogo Fatima	Yamoussoukro
Traoré Bakary	Bouaké

La sous-requête retourne la liste des id_client ayant acheté le forfait 3 (Forfait_internet). La requête principale récupère ensuite les noms.

2.3 Quand utiliser une sous-requête vs un JOIN ?

SITUATION	RECOMMANDATION
Filtrer selon une valeur calculée (moyenne, max...)	Sous-requête dans HAVING ou WHERE
Filtrer selon une liste d'IDs	Sous-requête avec IN
Afficher des colonnes des deux tables	JOIN
Réutiliser le même résultat plusieurs fois	CTE (WITH) → section suivante

💡 *Sous-requête ou JOIN produisent souvent le même résultat. En pratique, un JOIN est plus performant sur de grandes tables. Mais la sous-requête est parfois plus lisible pour exprimer un filtre.*

3. CTEs avec WITH — nommer ses étapes

Un **CTE** (Common Table Expression) permet de donner un nom à un résultat intermédiaire et de le réutiliser dans la même requête. C'est comme créer une table temporaire à la volée.

```
SQL
WITH nom_cte AS (
    SELECT ...      -- résultat intermédiaire
)
SELECT ...          -- requête principale qui utilise nom_cte
FROM nom_cte;
```

3.1 CTE simple — performance des agents

Question : Quels agents ont un CA $\geq$ 3 000 FCFA ?

```
SQL
WITH perf_agents AS (
    SELECT
        agent,
        COUNT(id_transaction) AS nb_ventes,
        SUM(montant)          AS ca
    FROM transactions
    GROUP BY agent
)
SELECT agent, nb_ventes, ca
FROM perf_agents
WHERE ca >= 3000
ORDER BY ca DESC;
```

AGENT	NB_VENTES	CA
Kouassi Éric	3	4 000
Koné Mamadou	3	4 000
Bamba Seydou	2	3 000

Sans CTE, il faudrait répéter le **GROUP BY** dans une sous-requête. Le CTE le fait une fois, et on peut y revenir autant de fois qu'on veut dans la suite de la requête.

3.2 CTE enchaînées — construire par étapes

Question : Rapport par mois — CA, évolution vs mois précédent.

On va construire ce rapport en **deux étapes** :

1. Calculer le CA par mois
2. Ajouter le CA du mois précédent avec **LAG** (on y revient en section 5)

SQL

```
WITH ca_par_mois AS (  
  SELECT  
    mois,  
    SUM(montant) AS ca  
  FROM transactions  
  GROUP BY mois  
) ,  
rapport_mensuel AS (  
  SELECT  
    mois,  
    ca,  
    LAG(ca) OVER (ORDER BY mois) AS ca_mois_precedent,  
    ca - LAG(ca) OVER (ORDER BY mois) AS evolution  
  FROM ca_par_mois  
)  
SELECT * FROM rapport_mensuel ORDER BY mois;
```

MOIS	CA	CA_MOIS_PRECEDENT	EVOLUTION
2024-01	5 500	NULL	NULL
2024-02	8 000	5 500	2 500
2024-03	2 000	8 000	-6 000

💡 Les CTE enchaînées se lisent de haut en bas. Chaque CTE peut utiliser les CTEs définies avant elle. C'est exactement comme organiser son travail en Excel : une feuille de calcul intermédiaire par étape, puis l'assemblage final.

Pont Excel → CTE

EXCEL	SQL AVEC CTE
Feuille intermédiaire "CA_Agents"	<code>WITH ca_agents AS (SELECT ...)</code>
Feuille finale qui y fait référence	<code>SELECT ... FROM ca_agents</code>
Nommer une plage	<code>WITH nom AS (...)</code>

Avantage CTE : pas besoin de créer de vraies tables ou feuilles. Tout existe le temps d'une requête, puis disparaît.

4. CASE WHEN — logique conditionnelle

`CASE WHEN` ajoute de la logique **if / else** directement dans SQL. On l'utilise pour créer des catégories, des libellés, ou des calculs conditionnels.

SQL

`CASE`

`WHEN condition1 THEN résultat1`

`WHEN condition2 THEN résultat2`

`ELSE résultat_par_défaut`

`END AS nom_colonne`

Pont Excel → CASE WHEN

Si tu viens d'Excel, `CASE WHEN` c'est exactement la fonction `SI()` imbriquée — ou `SI.CONDITIONS()` (IFS) dans les versions récentes.

EXCEL	SQL
<code>=SI(A2>=5000;"Gros";"Petit")</code>	<code>CASE WHEN ca >= 5000 THEN 'Gros' ELSE 'Petit' END</code>
<code>=SI.CONDITIONS(A2>=5000;"Gros";A2>=2500;"Moyen";VRAI;"Petit")</code>	<code>CASE WHEN ca>=5000 THEN 'Gros' WHEN ca>=2500 THEN 'Moyen' ELSE 'Petit' END</code>

💡 La règle de l'ordre s'applique aussi en Excel : dans un SI imbriqué, les conditions les plus restrictives doivent venir en premier.

4.1 Segmentation clients

Question : Classer chaque client en Petit / Moyen / Gros selon son CA total.

```
SQL
SELECT
  c.nom,
  SUM(t.montant) AS ca,
  CASE
    WHEN SUM(t.montant) >= 5000 THEN 'Gros client'
    WHEN SUM(t.montant) >= 2500 THEN 'Client moyen'
    ELSE 'Petit client'
  END AS segment
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.id_client, c.nom
ORDER BY ca DESC;
```


NOM	CA	SEGMENT
Kouassi Ama	5 000	Gros client
Traoré Bakary	2 500	Client moyen
Koné Mariama	2 000	Petit client
Diallo Ibrahim	2 000	Petit client
Yao Serge	2 000	Petit client
Ouédraogo Fatima	2 000	Petit client

4.2 CASE WHEN pour les agrégations conditionnelles

Question : Combien de transactions sont au-dessus de 1 000 FCFA vs en-dessous ?

SQL

```
SELECT
  COUNT(CASE WHEN montant > 1000 THEN 1 END) AS transactions_hautes,
  COUNT(CASE WHEN montant <= 1000 THEN 1 END) AS transactions_basses,
  SUM(CASE WHEN montant > 1000 THEN montant ELSE 0 END) AS ca_hautes
FROM transactions;
```

TRANSACTIONS_HAUTES	TRANSACTIONS_BASSES	CA_HAUTES
6	6	13 000



`COUNT(CASE WHEN ... THEN 1 END)` compte uniquement les lignes qui remplissent la condition. C'est l'équivalent SQL du **NB.SI** d'Excel.

4.3 Libellés lisibles

Question : Afficher le type de forfait en français lisible.

SQL

```
SELECT
  nom_forfait,
  prix,
  CASE type_forfait
    WHEN 'Recharge' THEN '📶 Recharge crédit'
    WHEN 'Forfait'   THEN '📦 Forfait data/voix'
    WHEN 'Pack'      THEN '👨‍👩‍👧 Pack famille'
    ELSE              type_forfait
  END AS libelle
FROM forfaits
ORDER BY prix;
```

NOM_FORFAIT	PRIX	LIBELLE
Recharge_500	500	📶 Recharge crédit
Forfait_data_nuit	800	📦 Forfait data/voix
Recharge_1000	1 000	📶 Recharge crédit
Forfait_voix	1 500	📦 Forfait data/voix
Forfait_internet	2 000	📦 Forfait data/voix
Pack_famille	3 500	👨‍👩‍👧 Pack famille

⚠️ Cette syntaxe `CASE colonne WHEN valeur` n'est possible que quand on compare une seule colonne à des valeurs fixes. Pour des conditions complexes (>, <, AND, OR), utilise la forme `CASE WHEN condition THEN`.

⚠️ Le piège de l'ordre des conditions

`CASE WHEN` s'arrête à la **première condition vraie** — les suivantes ne sont jamais évaluées.

SQL

```
-- ❌ Mauvais ordre : montant > 1500 sera toujours ignoré
CASE
  WHEN montant > 1000 THEN 'Élevé'
  WHEN montant > 1500 THEN 'Très élevé' -- jamais atteint !
  ELSE 'Faible'
END

-- ✅ Bon ordre : du plus restrictif au moins restrictif
CASE
  WHEN montant > 1500 THEN 'Très élevé'
  WHEN montant > 1000 THEN 'Élevé'
  ELSE 'Faible'
END
```

Règle : écris toujours les conditions de la plus grande valeur vers la plus petite.

5. Window functions — classements et comparaisons

Les **window functions** (fonctions de fenêtre) calculent une valeur pour chaque ligne **en relation avec d'autres lignes**, sans les fusionner comme **GROUP BY**.

SQL

```
FONCTION() OVER (
  PARTITION BY colonne -- facultatif : sous-groupes
  ORDER BY colonne     -- ordre de calcul
)
```

💡 La différence clé avec **GROUP BY** : les window functions ajoutent une colonne calculée à chaque ligne existante. **GROUP BY** réduit le nombre de lignes.

5.1 RANK() — classement avec ex-aequo

Question : Classer les agents par chiffre d'affaires.

SQL

```
SELECT
  agent,
  SUM(montant) AS ca,
  RANK() OVER (ORDER BY SUM(montant) DESC) AS rang
FROM transactions
GROUP BY agent
ORDER BY rang;
```

AGENT	CA	RANG
Kouassi Éric	4 000	1
Koné Mamadou	4 000	1
Bamba Seydou	3 000	3
Traoré Aminata	2 500	4
N'Guessan Fatou	2 000	5

`RANK()` gère les **ex-aequo** : deux agents à 4 000 FCFA reçoivent le rang 1, et le suivant reçoit le rang 3 (pas 2). Pour éviter les sauts, utilise `DENSE_RANK()`.

5.2 ROW_NUMBER() — numéroté dans chaque groupe

Question : Numéroté les transactions par ordre de montant décroissant, mois par mois.

SQL

```
SELECT
  mois,
  montant,
  ROW_NUMBER() OVER (
    PARTITION BY mois
    ORDER BY montant DESC
  ) AS rang_dans_mois
FROM transactions
ORDER BY mois, rang_dans_mois;
```

MOIS	MONTANT	RANG_DANS_MOIS
2024-01	2 000	1
2024-01	1 500	2
2024-01	1 000	3
2024-01	500	4
2024-01	500	5
2024-02	2 000	1
2024-02	2 000	2
...	...	...

`PARTITION BY mois` repart le compteur à 1 à chaque nouveau mois. Sans `PARTITION BY`, `ROW_NUMBER()` numéroteait toutes les lignes en continu.

5.3 LAG() — accéder à la ligne précédente

Question : Rapport mensuel avec CA du mois précédent et évolution.

SQL

```
WITH ca_par_mois AS (  
  SELECT mois, SUM(montant) AS ca  
  FROM transactions  
  GROUP BY mois  
)  
SELECT  
  mois,  
  ca,  
  LAG(ca) OVER (ORDER BY mois) AS ca_precedent,  
  ca - LAG(ca) OVER (ORDER BY mois) AS evolution  
FROM ca_par_mois  
ORDER BY mois;
```

MOIS	CA	CA_PRECEDENT	EVOLUTION
2024-01	5 500	NULL	NULL
2024-02	8 000	5 500	+2 500
2024-03	2 000	8 000	-6 000

`LAG(ca)` retourne la valeur de `ca` de la ligne **précédente** selon l'ORDER BY. La première ligne n'a pas de précédent → `NULL`. `LEAD()` fait l'inverse : il regarde la ligne **suivante**.

Récapitulatif window functions

FONCTION	CE QU'ELLE FAIT	CAS D'USAGE
<code>RANK()</code>	Rang avec sauts sur ex-aequo (1,1,3)	Podium agents, top clients
<code>DENSE_RANK()</code>	Rang sans sauts (1,1,2)	Classement serré
<code>ROW_NUMBER()</code>	Numéro unique par ligne	Pagination, top-N par groupe
<code>LAG(col)</code>	Valeur de la ligne précédente	Comparaison mois N vs N-1
<code>LEAD(col)</code>	Valeur de la ligne suivante	Anticipation, prochaine valeur

5.4 DENSE_RANK() vs RANK() — comparaison directe

SQL

```
SELECT
  agent,
  SUM(montant) AS ca,
  RANK() OVER (ORDER BY SUM(montant) DESC) AS rang_rank,
  DENSE_RANK() OVER (ORDER BY SUM(montant) DESC) AS rang_dense
FROM transactions
GROUP BY agent
ORDER BY ca DESC;
```

AGENT	CA	RANG_RANK	RANG_DENSE
Kouassi Éric	4 000	1	1
Koné Mamadou	4 000	1	1
Bamba Seydou	3 000	3	2
Traoré Aminata	2 500	4	3
N'Guessan Fatou	2 000	5	4

RANK() saute le rang 2 après l'ex-aequo → Bamba Seydou est 3ème.

DENSE_RANK() ne saute pas → Bamba Seydou est 2ème.

Choix : utilise **RANK()** pour un vrai podium sportif (1er, 1er, 3ème), **DENSE_RANK()** quand tu veux un classement continu sans trous.

5.5 LEAD() — regarder la ligne suivante

LAG() regarde en arrière, **LEAD()** regarde en avant. Utile pour afficher la prochaine valeur dans une chronologie.

SQL

```
WITH ca_par_mois AS (  
  SELECT mois, SUM(montant) AS ca  
  FROM transactions  
  GROUP BY mois  
)  
SELECT  
  mois,  
  ca,  
  LEAD(ca) OVER (ORDER BY mois) AS ca_mois_suivant  
FROM ca_par_mois  
ORDER BY mois;
```

MOIS	CA	CA_MOIS_SUIVANT
2024-01	5 500	8 000
2024-02	8 000	2 000
2024-03	2 000	NULL

La dernière ligne n'a pas de suivant → **NULL**. **LEAD()** est utile pour anticiper : afficher la prochaine échéance, le prochain mois cible, etc.

6. Tout combiner — le rapport mensuel complet

On assemble maintenant tous les outils du module en une seule requête : le rapport mensuel demandé par le DG.

Objectif : pour chaque agent, afficher son CA, son rang, et son segment de performance.

SQL

```
WITH perf AS (
  SELECT
    t.agent,
    t.region,
    COUNT(t.id_transaction) AS nb_ventes,
    SUM(t.montant) AS ca
  FROM transactions t
  GROUP BY t.agent, t.region
),
rapport AS (
  SELECT
    agent,
    region,
    nb_ventes,
    ca,
    RANK() OVER (ORDER BY ca DESC) AS rang,
    CASE
      WHEN ca >= 4000 THEN '🟢 Top performer'
      WHEN ca >= 3000 THEN '🟡 Bon agent'
      ELSE '🔴 À accompagner'
    END AS statut
  FROM perf
)
SELECT * FROM rapport ORDER BY rang;
```

AGENT	REGION	NB_VENTES	CA	RANG	STATUT
Kouassi Éric	Abidjan	3	4 000	1	🟢 Top performer
Koné Mamadou	Abidjan	3	4 000	1	🟢 Top performer
Bamba Seydou	Yamoussoukro	2	3 000	3	🟡 Bon agent
Traoré Aminata	Bouaké	2	2 500	4	🔴 À accompagner
N'Guessan Fatou	San Pedro	2	2 000	5	🔴 À accompagner

🎯 Ce rapport utilise : CTE (perf → rapport), window function (RANK), CASE WHEN (statut), GROUP BY (agrégation). C'est le type de requête qu'un DA junior livre au management chaque fin de mois.

6.5 Exercice pratique — Mets en pratique

Les données sont déjà chargées dans sqliteonline.com. Écris les requêtes toi-même.

Exercice A — Sous-requête : forfaits au-dessus du prix moyen

Affiche les forfaits dont le prix est supérieur au prix moyen du catalogue.

👉 Voir la solution

```
SQL
SELECT nom_forfait, prix, type_forfait
FROM forfaits
WHERE prix > (SELECT AVG(prix) FROM forfaits)
ORDER BY prix DESC;
```

NOM_FORFAIT	PRIX	TYPE_FORFAIT
Pack_famille	3 500	Pack
Forfait_internet	2 000	Forfait
Forfait_voix	1 500	Forfait

La moyenne des prix : $(500+1000+2000+1500+800+3500)/6 = 1\,550$ FCFA.

Exercice B — CASE WHEN : catégoriser les transactions

Pour chaque transaction, ajoute une colonne `taille` : "Petite" si montant < 1 000, "Moyenne" si entre 1 000 et 1 999, "Grande" si $\geq 2\ 000$.

👉 Voir la solution

SQL

```
SELECT
  id_transaction,
  agent,
  montant,
  CASE
    WHEN montant >= 2000 THEN 'Grande'
    WHEN montant >= 1000 THEN 'Moyenne'
    ELSE 'Petite'
  END AS taille
FROM transactions
ORDER BY montant DESC;
```

ID_TRANSACTION	AGENT	MONTANT	TAILLE
1	Koné Mamadou	2 000	Grande
6	Bamba Seydou	2 000	Grande
...	...	...	...
2	Traoré Aminata	500	Petite

Exercice C — CTE + RANK : top 3 clients

Affiche les 3 meilleurs clients par CA, avec leur rang.

👉 Voir la solution

SQL

```
WITH classement AS (  
  SELECT  
    c.nom,  
    c.ville,  
    SUM(t.montant) AS ca,  
    RANK() OVER (ORDER BY SUM(t.montant) DESC) AS rang  
  FROM transactions t  
  JOIN clients c ON t.id_client = c.id_client  
  GROUP BY c.id_client, c.nom, c.ville  
)  
SELECT * FROM classement WHERE rang <= 3 ORDER BY rang;
```

NOM	VILLE	CA	RANG
Kouassi Ama	Abidjan	5 000	1
Traoré Bakary	Bouaké	2 500	2
Koné Mariama	San Pedro	2 000	3

📌 `rang <= 3` dans le *WHERE* de la requête principale — on ne peut pas filtrer directement sur une window function, d'où le passage par CTE.

7. L'IA pour écrire du SQL — méthode et limites

Les outils d'IA comme **Claude** ou **ChatGPT** peuvent générer du SQL en quelques secondes. Bien utilisés, ils te font gagner un temps considérable. Mal utilisés, ils produisent du SQL qui a l'air correct mais donne de faux résultats.

7.1 La méthode : contexte → question → vérification

Un bon prompt SQL suit toujours le même schéma :

1. Décris tes tables (noms, colonnes clés)
2. Pose ta question business en français
3. Précise le moteur SQL (SQLite, PostgreSQL, BigQuery...)
4. Demande une explication ligne par ligne

Exemple de bon prompt :

```
J'ai 3 tables SQLite :  
- clients(id_client, nom, ville, date_inscription)  
- transactions(id_transaction, id_client, id_forfait, agent, region, montant, mois)  
- forfaits(id_forfait, nom_forfait, prix, type_forfait)  
  
Question : pour chaque ville, donne-moi le nombre de clients distincts  
ayant effectué une transaction en 2024-02, et leur CA total.  
Explique chaque ligne de la requête.
```

Ce que l'IA va produire (et comment le lire) :

```
SQL  
SELECT  
  c.ville,  
  COUNT(DISTINCT c.id_client) AS nb_clients,  -- clients uniques, pas  
    transactions  
  SUM(t.montant)                AS ca  
FROM transactions t  
JOIN clients c ON t.id_client = c.id_client  
WHERE t.mois = '2024-02'  
GROUP BY c.ville  
ORDER BY ca DESC;
```

7.2 Les 4 erreurs classiques de l'IA en SQL

ERREUR	CE QUE L'IA FAIT	COMMENT DÉTECTER
Hallucination de colonnes	Invente une colonne qui n'existe pas	Erreur à l'exécution
Mauvaise jointure	Joint sur la mauvaise clé (ex: montant = prix)	Résultat trop grand
NULL ignoré	Oublie <code>COALESCE</code> ou <code>IS NULL</code>	Lignes manquantes
Syntaxe moteur	Écrit <code>FULL OUTER JOIN</code> pour SQLite	Erreur à l'exécution

7.3 Les règles d'or avec l'IA

✅ Ce que l'IA fait bien :

- Générer le squelette d'une requête complexe
- Convertir une question en SQL (`GROUP BY`, `JOIN` simples)
- Expliquer une requête que tu n'as pas écrite
- Déboguer une erreur de syntaxe

❌ Ce que tu dois toujours faire toi-même :

- **Vérifier les résultats** — compare avec des données que tu connais
- **Tester sur un sous-ensemble** — d'abord `LIMIT 5`, puis sans limite
- **Valider la logique métier** — l'IA ne connaît pas ton contexte ANSUT-CI

7.4 Exercice IA — débogage

L'IA a généré cette requête. Trouve les 2 erreurs :

```
SQL
SELECT ville, SUM(montant) AS ca
FROM transactions
JOIN clients ON id_client = id_client
GROUP BY ville
ORDER BY ca;
```

👉 Voir les erreurs

Erreur 1 : `ON id_client = id_client` est ambigu — les deux tables ont `id_client`. Il faut `ON t.id_client = c.id_client`.

Erreur 2 : `ville` dans le `SELECT` vient de `clients`, mais sans alias ni préfixe, c'est ambigu. Et `ville` dans le `GROUP BY` est également ambigu. Il faut `c.ville`.

Requête corrigée :

```
SQL
SELECT c.ville, SUM(t.montant) AS ca
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.ville
ORDER BY ca DESC;
```

💡 *L'IA est un co-pilote, pas un pilote automatique. Elle te fait gagner du temps sur la syntaxe — c'est toi qui garantis la qualité du résultat.*

8. Cheatsheet — SQL Reporting

Sous-requêtes

```
SQL
-- Scalaire : filtrer selon une valeur calculée
WHERE col > (SELECT AVG(col) FROM table)

-- Liste : filtrer selon un ensemble d'IDs
WHERE id IN (SELECT id FROM autre_table WHERE condition)
```

CTEs

```
SQL
WITH nom1 AS (
  SELECT ...
),
nom2 AS (
  SELECT ... FROM nom1  -- peut réutiliser nom1
)
SELECT ... FROM nom2;
```

CASE WHEN

```
SQL
CASE
  WHEN condition1 THEN valeur1
  WHEN condition2 THEN valeur2
  ELSE valeur_defaut
END AS nom_colonne

-- Forme courte (égalité simple)
CASE colonne
  WHEN 'A' THEN 'Libellé A'
  WHEN 'B' THEN 'Libellé B'
END
```

Window functions

```
SQL
RANK()          OVER (ORDER BY col DESC)      -- rang avec sauts
DENSE_RANK()    OVER (ORDER BY col DESC)      -- rang sans sauts
ROW_NUMBER()    OVER (PARTITION BY grp ORDER BY col)  -- numéro unique par
groupe
LAG(col)        OVER (ORDER BY col)           -- valeur ligne
précédente
LEAD(col)       OVER (ORDER BY col)           -- valeur ligne suivante
```


Ordre d'exécution SQL complet

FROM + JOIN → WHERE → GROUP BY → HAVING → SELECT (+ window) → ORDER BY
→ LIMIT

Les window functions s'exécutent **après** **SELECT** — c'est pourquoi elles peuvent référencer des alias définis dans le **SELECT**.

Prompt IA efficace

```
Tables : [liste tes tables et colonnes clés]
Moteur : SQLite / PostgreSQL / BigQuery
Question : [question business en français]
→ Donne la requête + une explication ligne par ligne
```

9. Quiz — Vérifie ta compréhension

Q1. Quelle est la différence principale entre une sous-requête et un CTE ?

- a) La sous-requête est plus rapide
- b) Le CTE est réutilisable plusieurs fois dans la même requête ; la sous-requête ne l'est pas
- c) Le CTE ne peut pas utiliser JOIN

 [Voir la réponse](#)

 **b)** — Un CTE peut être référencé plusieurs fois dans la requête principale. Une sous-requête en ligne doit être répétée à chaque usage. Les CTEs rendent aussi le code beaucoup plus lisible.

Q2. Tu veux afficher les produits dont le prix est supérieur au prix moyen. Quelle requête est correcte ?

```
SQL
-- Option A
SELECT nom_forfait FROM forfaits
WHERE prix > AVG(prix);

-- Option B
SELECT nom_forfait FROM forfaits
WHERE prix > (SELECT AVG(prix) FROM forfaits);
```

👉 Voir la réponse

✅ **Option B** — `AVG()` est une fonction d'agrégation, elle ne peut pas être utilisée directement dans `WHERE`. Il faut la calculer dans une sous-requête. L'option A produit une erreur SQL.

Q3. Que retourne `RANK()` si deux lignes ont la même valeur ?

- a) Une erreur
- b) Le même rang pour les deux, et le rang suivant saute (1, 1, 3)
- c) Des rangs différents attribués arbitrairement

👉 Voir la réponse

✅ **b)** — `RANK()` attribue le même rang aux ex-aequo et saute le rang suivant. Si tu veux éviter les sauts, utilise `DENSE_RANK()` qui donne 1, 1, 2.

Q4. Quelle est la différence entre `RANK()` et `ROW_NUMBER()` sur des ex-aequo ?

- a) Aucune, ils retournent la même chose
- b) `ROW_NUMBER()` attribue un numéro unique à chaque ligne, même en cas d'ex-aequo ;
`RANK()` attribue le même rang
- c) `ROW_NUMBER()` ne fonctionne qu'avec `PARTITION BY`

👉 Voir la réponse

✓ b) — Sur deux lignes avec la même valeur : `RANK()` → 1, 1 (puis 3) ; `ROW_NUMBER()` → 1, 2 (ordre arbitraire entre les ex-aequo). `ROW_NUMBER()` garantit l'unicité.

Q5. À quoi sert `PARTITION BY` dans une window function ?

- a) À filtrer les lignes comme `WHERE`
- b) À diviser les données en groupes pour que la window function repart à zéro dans chaque groupe
- c) À remplacer `GROUP BY`

👉 Voir la réponse

✓ b) — `PARTITION BY mois` dans un `ROW_NUMBER()` fait repartir le compteur à 1 pour chaque mois. Sans `PARTITION BY`, la window function s'applique sur toutes les lignes d'un coup.

Q6. Tu veux la valeur du mois précédent dans un rapport mensuel. Quelle fonction utilises-tu ?

- a) `PREV()`
- b) `LAG(colonne) OVER (ORDER BY mois)`
- c) `SHIFT(colonne, -1)`

👉 Voir la réponse

✓ b) — `LAG(col) OVER (ORDER BY mois)` retourne la valeur de `col` à la ligne précédente selon l'ordre des mois. La première ligne retourne `NULL`. `LEAD()` fait l'inverse (ligne suivante).

Q7. Cette requête CASE WHEN a une erreur. Laquelle ?

SQL

```
SELECT nom, montant,  
  CASE  
    WHEN montant > 1000 THEN 'Élevé'  
    WHEN montant > 500  THEN 'Moyen'  
    WHEN montant > 1500 THEN 'Très élevé'  
    ELSE 'Faible'  
  END AS niveau  
FROM transactions;
```

👉 Voir la réponse

✅ La condition `montant > 1500` ne sera jamais atteinte — les lignes avec `montant > 1500` sont déjà capturées par `montant > 1000` au-dessus. `CASE WHEN` s'arrête à la **première condition vraie**. Il faut mettre les conditions dans l'ordre du plus restrictif au moins restrictif :

SQL

```
CASE  
  WHEN montant > 1500 THEN 'Très élevé'  
  WHEN montant > 1000 THEN 'Élevé'  
  WHEN montant > 500  THEN 'Moyen'  
  ELSE 'Faible'  
END
```

Q8. Quel est le principal risque quand on utilise l'IA pour générer du SQL ?

- a) L'IA ne peut pas écrire de JOIN
- b) L'IA génère une syntaxe parfaite mais une logique incorrecte — les résultats semblent plausibles mais sont faux
- c) L'IA est trop lente pour être utile

👉 Voir la réponse

✅ **b)** — C'est le risque principal : le SQL s'exécute sans erreur, mais la logique (mauvaise jointure, mauvaise clé, oubli d'un filtre) donne des résultats incorrects. Sans vérification manuelle sur des données connues, l'erreur passe inaperçue.

Q9. Tu veux le top 3 des clients par CA. Quelle approche est correcte en SQLite ?

- a) `WHERE RANK() <= 3`
- b) CTE avec `RANK()`, puis `WHERE rang <= 3` dans la requête principale
- c) `LIMIT 3` sans `ORDER BY`

👉 Voir la réponse

✅ **b)** — Les window functions ne peuvent pas être filtrées directement dans `WHERE` (elles s'exécutent après). Il faut passer par un CTE ou une sous-requête :

```
SQL
WITH classement AS (
  SELECT c.nom, SUM(t.montant) AS ca,
         RANK() OVER (ORDER BY SUM(t.montant) DESC) AS rang
  FROM transactions t JOIN clients c ON t.id_client = c.id_client
  GROUP BY c.id_client
)
SELECT * FROM classement WHERE rang <= 3;
```

Le c) est dangereux — sans `ORDER BY`, `LIMIT` retourne des lignes arbitraires.

Q10. Quelle est la bonne séquence pour utiliser l'IA sur une requête SQL complexe ?

- a) Copier-coller la requête IA directement en production
- b) Donner le contexte (tables, colonnes) → recevoir la requête → l'exécuter sur des données test → vérifier les résultats → ajuster si besoin
- c) Demander à l'IA de vérifier ses propres résultats

👉 Voir la réponse

✅ **b)** — La méthode correcte : contexte précis → requête → test sur données connues → vérification → ajustement. L'IA ne peut pas vérifier ses propres résultats (elle ne voit pas ta base de données réelle). Toujours tester manuellement.

10. Résumé du module

OUTIL	CE QU'IL FAIT	QUAND L'UTILISER
Sous-requête scalaire	Valeur calculée dans WHERE/HAVING	Comparer à une moyenne, un max
Sous-requête IN	Liste d'IDs dans WHERE	Filtrer selon une autre table
CTE (<code>WITH</code>)	Résultat intermédiaire nommé	Requêtes en plusieurs étapes, réutilisation
<code>CASE WHEN</code>	Logique conditionnelle	Segments, libellés, calculs conditionnels
<code>RANK()</code>	Rang avec sauts sur ex-aequo	Podium, classement avec égalités
<code>DENSE_RANK()</code>	Rang sans sauts	Classement sans trous
<code>ROW_NUMBER()</code>	Numéro unique par ligne	Top-N par groupe, pagination
<code>LAG()</code> / <code>LEAD()</code>	Valeur ligne précédente / suivante	Comparaison temporelle
IA pour SQL	Génère du SQL depuis une question	Gain de temps — toujours vérifier

✅ Acte III — SQL : ce que tu sais maintenant

Tu as parcouru les 4 modules de l'Acte SQL :

MODULE	COMPÉTENCES
M06	SELECT, WHERE, ORDER BY, LIMIT — les bases
M07	COUNT, SUM, AVG, GROUP BY, HAVING — les agrégations
M08	JOIN, LEFT JOIN, anti-join, FULL OUTER JOIN — les jointures
M09	Sous-requêtes, CTE, CASE WHEN, window functions, IA — le reporting

Tu es capable d'écrire des requêtes de reporting réelles, telles qu'un DA junior les produit en entreprise.

➡ **Acte IV — Statistiques & Code**

Dans l'**Acte IV**, on quitte SQL pour entrer dans le monde du code :

- **Module 10** — Environnements R & Python : installation, premiers pas, Jupyter vs RStudio
- **Module 11** — Statistiques descriptives : moyenne, médiane, quartiles, distribution
- **Module 12** — Visualisation : graphiques avec ggplot2 (R) et matplotlib / seaborn (Python)

→ **Module 10 — Environnements R & Python**